Color Space Conversion

SENG 440 Embedded Systems

Matt Smith - matthewsmith@uvic.ca (V00966014)

Brendon Waters - brendonwaters@uvic.ca (V00963713)

Table of contents

Project Design

SIMD Architecture & Vector Registers:

- Leveraged ARM NEON 128-bit vector registers to process multiple pixel channels simultaneously.
- Utilized vector types including uint8x16_t, uint16x8_t, and int32x4_t for wide loading and accumulation.

Fixed-Point Specification:

- Implemented a Q8 fixed-point format (K=8, scaling factor 2^8 = 256) to perform color conversions without floating-point hardware.

Test Datasets:

- Evaluated pipeline performance across two PPM image resolutions: 64x48 (small) and 640x480 (large).

Quality Constraints:

- Maintained high conversion fidelity, targeting a mean absolute error around 1.85 levels (scale 0–255) across full round-trip execution.

Project Environment

Target Hardware & Platforms:

- Primary Target: University of Victoria Single Board Computer (SBC) running ARMv7-A.
- Emulation Environment: 32-bit QEMU emulator on a Linux Virtual Machine.
- Comparative Benchmarking: Raspberry Pi 3B v1.2 (Broadcom BCM2837 Cortex-A53, ARMv8-A architecture).

Compilation & Flags:

- Compiled via arm-none-linux-gnueabihf-gcc -march=armv7-a -mfpu=neon -mfloat-abi=hard
- Evaluated compiler output at -O0 and -O2 optimization levels.

Background

RGB vs. YCbCr Color Spaces:

- **RGB:** Represents raw red, green, and blue intensity per channel (0, 255).
- **YCbCr:** Separates luminance (16, 235) from chrominance difference channels (16, 240).
- **Motivation:** Human vision is significantly more sensitive to brightness variations than color variations.

4:2:0 Chroma Subsampling:

- Images are divided into 2x2 pixel blocks.
- Full resolution is maintained for luminance Y, while Cb and Cr are averaged across all 4 pixels down to 1 sample.
- **Downsampling:** Averages 2x2 color values during RGB to YCC.
- **Upsampling:** Interpolates missing color channels for reconstructed pixels during YCC to RGB.

Background

Fixed-Point Arithmetic:

- Embedded targets lacking a dedicated Floating-Point Unit (FPU) incur severe performance penalties when emulating floating-point operations.
- Integer-based fixed-point arithmetic drastically speeds up calculations using integer MAC units.

Precision Trade-Offs (K=8):

- Coefficients are pre-multiplied by 2^8 = 256 and rounded to integers.
- K=8 provides sufficient numerical accuracy while keeping intermediate sums within register capacity.
- Higher precision (K=16) would cause intermediate products to overflow 16-bit accumulators during matrix operations.

Verification and Profiling

Profiling Methodology:

- **Primary Metric:** Valgrind / Cachegrind dynamic instruction counts and L1/Last-Level (LL) cache misses to eliminate VM timing distortions.
- **Secondary Metric:** CLOCK_MONOTONIC wall-clock time (averaged over multiple iterations post-warmup).

Test Execution Workflow:

- Load PPM image into global RGB memory arrays and retain an unmodified reference snapshot.
- Perform an initial untimed warm up run to prime CPU data caches.
- Execute timed iterations: convert RGB to YCC to RGB, recording stage-specific timing.

Quality & Correctness Tracking:

- Evaluated reconstructed RGB data against snapshot images using maximum channel difference (diff_max) and mean absolute channel error (mean_abs_delta).
- Automated testing and CSV profiling via unified make measure tooling.

Issues Encountered

Correctness & Performance Bug Fixes:

- **Upsampling Loop Placement:** Fixed redundant chrominance_array_upsample calls inside nested pixel loops, reducing baseline YCC to RGB instructions from 17.39M to 193k.
- **Coefficient Scale Fix:** Corrected 6-bit YCC coefficients to 8-bit, resolving a 4x darkening artifact in output images.
- **Accumulator Overflow:** Resolved wrap-around black pixel artifacts by upgrading vector accumulators to 32-bit registers.

Algorithm Design

RGB to YCbCr (fixed-point, K=8):

- Y = ((16<<K) + C11*R + C12*G + C13*B + bias) >> K
- Cb = ((128<<K) - C21*R - C22*G + C23*B + bias) >> K
- Cr = ((128<<K) + C31*R - C32*G - C33*B + bias) >> K
- Coefficients scaled by 2^K=256:
    - C11=66, C12=129, C13=25, C21=38, C22=74, C23=112, C31=112, C32=94, C33=18

YCbCr to RGB (fixed-point):

- R = (D1*(Y-16) + D2*(Cr-128) + bias) >> K
- G = (D1*(Y-16) - D3*(Cr-128) - D4*(Cb-128) + bias) >> K
- B = (D1*(Y-16) + D5*(Cb-128) + bias) >> K
    - Coefficients: D1=298, D2=409, D3=208, D4=100, D5=516

Rounding vs. truncation trade-off:

- Bias term (1<<(K-1)) before the final shift rounds instead of truncates, removing a systematic darkening error.
- This precision costs register headroom - the pre-shift sum needs double the bit-width of the final 8-bit result, which is why our accumulators grew from 16-bit to 32-bit later on.

Code Optimizations

Initial NEON Implementation:

- We used SIMD vectors to perform the Multiply-Load-Accumulate operations simultaneously across all three YCC channels but we were only travering 2x2 pixels at a time. We placed these 4 bytes into `int32x4_t` which left a lot of unused space in the register.

Tiling:

- Then we restructured the outer loop to fetch 16x16 tiles into memory but still only processed 2x2 pixels at a time which didn't provide significant speedup.

LUT:

- Another implementation attempt that resulted in little to no speedup by trying to precompute the multiplications but the number of instructions for loading the data was significantly worse

neon_v2():

- Changed the outer loop to traverse/fetch 16 columns and 2 rows at a time enabling us to process 4 pixel collections at once.
- Used `vld1_u8` to load all the pixels into the register at once
- Used `vmovl_u8` to widen each 8-bit (`uint8x8_t`) section into 16-bit (`uint16x8_t`)
- Resulted in ~2.5x fewer instructions across the entire image processing

Code Optimizations

neon_v3:

- After porting our _v2 solution to the inverse YCC-to-RGB code we discovered that due to the size of the coefficients that we were seeing integer overflow. This resulted in a mostly black image.
- D1*(Y-16) + D2*(Cr-128) can reach ~123,165 a value larger than (2^15) - 1 = 32,767, the max signed integer value for `int16_t`
- Solved by splitting each 8 lane value into two halves. Low and high. We then load the data into two different 128-bit NEON registers and perform the typical Multiply-Add-Accumulate operations without overflow
- The final values for R, G and B are then narrowed back to 8 bits each using the `vqmovun` intrinsic. The q signals that this is a saturating move, to ensure that we clamp our result between 0 and 255.

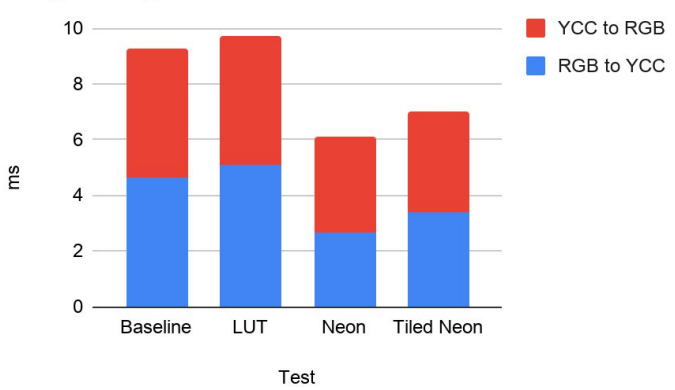
Compiler & Assembly Analysis

- Compiled each routine with `arm-none-linux-gnueabihf-gcc -march=armv7-a -mfpu=neon -mfloat-abi=hard -S` with both `-O0` and `-O2` flags to compare the compiler optimizations
- Notably, the neon_v2 Assembly loop as more instructions that then integer implementation but when we step out and count instructions across the entire image processing NEON significantly improves.

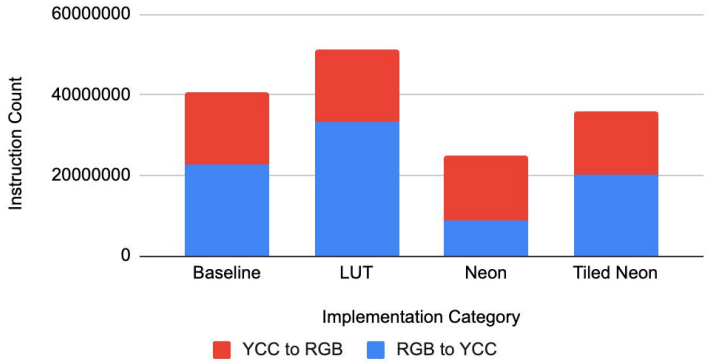
RGB to YCC only	integer	neon_v2
Main loop body	216	279
Full image	~22 million	~8 million
	-	2.5x fewer

Profiling Results

Wall-time for large image conversion



Instruction count for large image conversion



Profiling Results

Wall-Clock Time Reductions:

- **Small Images (64x48):** Total runtime decreased from 0.100ms to 0.060ms (**~40% speedup**).
- **Large Images (640x480):** Total runtime decreased from 9.25ms to 6.12ms (**~34% speedup**).

Instruction Count Performance (Cachegrind):

- **RGB to YCC (Large Image):** Instruction count dropped from 22.6M to 8.9M (**~2.5x reduction**).
- **YCC to RGB (Large Image):** Instruction count dropped from 17.9M to 16.0M (**~1.12x reduction**).

Cache & Quality Findings:

- Last-Level (LL) cache misses remained flat across implementations (~273,594 for large images), demonstrating that computational throughput (not cache misses) was the primary bottleneck.
- Average round-trip RGB mean error remained locked at 1.85 across all test variants, proving speedups did not degrade image quality.

Conclusion

Key Takeaways:

- Fixed-point arithmetic (K=8) combined with wide NEON strip processing delivered the highest performance gains.
- Achieved a **2.5x reduction in instruction count** while maintaining strict visual fidelity (1.85 mean RGB error).

Insights:

- Hypothesized optimizations (Lookup Tables and 16x16 Tiling) degraded performance due to cache traffic and loop management overheads.
- Profiling with Cachegrind and wall-clock benchmarks proved essential in finding true bottlenecks vs. intuitive assumptions.

Future Optimization Opportunities:

- Vectorizing the chroma upsampling interpolation step directly to reduce YCC to RGB overhead.
- Optimizing intermediate memory buffer transfers to minimize row/column traversal delays.